



数据库系统

第五章 数据库完整性

胡金鹏



■ 数据库的完整性

数据的**正确性**

- 是指数据是符合现实世界语义，反映了当前实际状况的

数据的**相容性**

- 是指数据库同一对象在不同关系表中的数据是符合逻辑

例如，

- 学生的学号必须唯一
- 性别只能是男或女
- 本科学生年龄的取值范围为14~50的整数
- 学生所选的课程必须是学校开设的课程，学生所在的院系必须是学校已成立的院系



■ 为什么要由数据库控制完整性，而不用应用程序来实现完整性控制？

- **不正当的数据库操作，如输入错误、操作失误、程序处理失误等**

由应用程序来实现完整性控制是有漏洞的。有的应用程序定义的完整性约束条件可能被其他应用程序破坏，造成数据库数据的正确性无法保障。所以，需要使得完整性控制成为DBMS核心支持的功能，为所有的用户和所有的应用提供一致的数据库完整性。

■ 数据库完整性管理的作用

- **防止和避免数据库中不合理数据的出现**
- **DBMS应尽可能地自动防止DB中语义不合理现象**

如DBMS不能自动防止，则需要应用程序员和用户在数据库操作时处处加以小心，每写一条SQL语句都要考虑是否符合语义完整性，这种工作负担是非常沉重的，因此应尽可能多地让DBMS来承担。



为维护数据库的完整性，DBMS必须：

■1.提供定义完整性约束条件的机制

完整性约束条件也称为完整性规则，是数据库中的数据必须满足的语义约束条件。SQL标准通过数据定义语言来描述完整性，包括关系模型的实体完整性、参照完整性和用户定义的完整性

■2.提供完整性检查的方法

数据库管理系统重检查数据是否满足完整性约束条件的机制称为完整性检查。一般在INSERT、UPDATE、DELETE语句后开始完整性检查，也可以在事务提交时检查

■3.违约处理

数据库管理系统若发现用户的操作违背了完整性约束条件，就采取拒绝（NO ACTION执行该操作或级联（CASCADE）执行其他操作等方式保证完整性



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- 关系模型的**实体完整性**
 - 设置主码，让每条记录相互可区分，CREATE TABLE中用PRIMARY KEY定义
- **单属性**构成的码有两种说明方法
 - 定义为列级约束条件
 - 定义为表级约束条件
- 对**多个属性**构成的码只有一种说明方法
 - 定义为表级约束条件



【例1】将Student表中的Sno属性定义为码

(1)在列级定义主码

CREATE TABLE Student

(Sno CHAR(9) PRIMARY KEY ,

Sname CHAR(20) NOT NULL ,

Ssex CHAR(2) ,

Sage SMALLINT ,

Sdept CHAR(20));



【例1】将Student表中的Sno属性定义为码

(2)在表级定义主码

```
CREATE TABLE Student  
(Sno CHAR(9) ,  
Sname CHAR(20) NOT NULL ,  
Ssex CHAR(2) ,  
Sage SMALLINT ,  
Sdept CHAR(20) ,  
PRIMARY KEY (Sno)  
);
```




【例2】将SC表中的Sno , Cno属性组定义为码

(3)只能在表级定义主码

CREATE TABLE SC

(Sno CHAR(9) NOT NULL ,

Cno CHAR(4) NOT NULL ,

Grade SMALLINT ,

PRIMARY KEY (Sno, Cno)

);



插入或对主码列进行更新操作时，关系数据库管理系统按照实体完整性规则自动进行检查。包括：

- 检查主码值是否唯一，如果不唯一则拒绝插入或修改
- 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改

检查主码值是否唯一的两种方法：

- 全表扫描
- 索引



- 检查记录中主码值是否唯一的一种方法是进行全表扫描
依次判断表中每一条记录的主码值与将插入记录上的主码值（或者修改的新主码值）是否相同
- 表扫描缺点：十分耗时
- 为避免对基本表进行全表扫描，RDBMS核心一般都在主码上自动建立一个索引

待插入记录

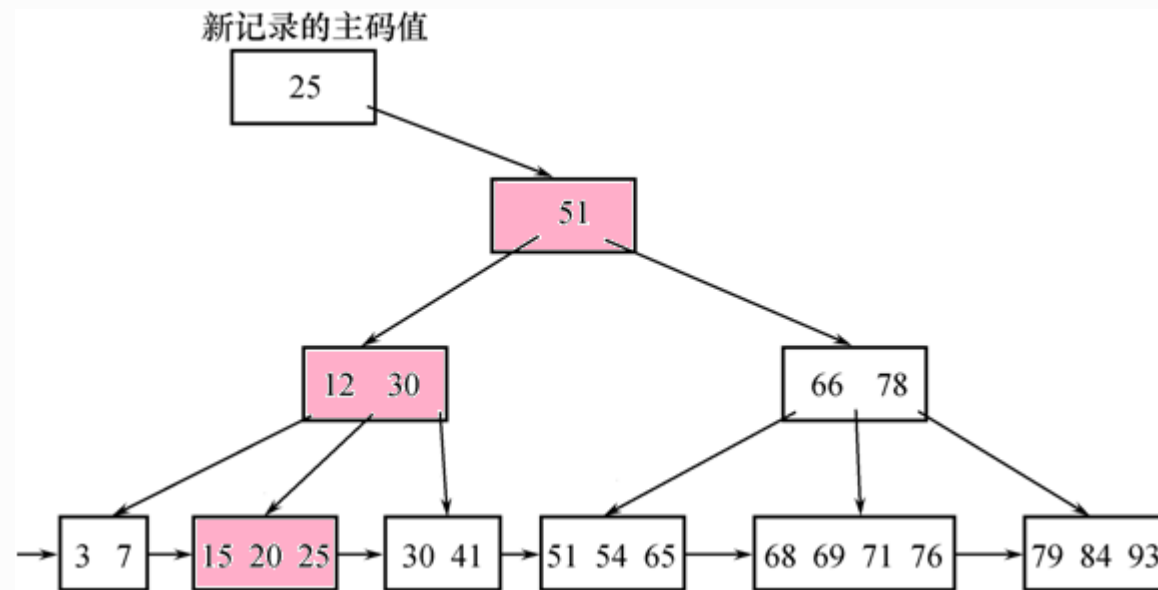
Key _i	F2 _i	F3 _i	F4 _i	F5 _i
------------------	-----------------	-----------------	-----------------	-----------------

基本表

Key1	F21	F31	F41	F51
Key2	F22	F32	F42	F52
Key3	F23	F33	F43	F53
⋮				



• B+树索引



例如，

- 新插入记录的主码值是25
 - 通过主码索引，从B+树的根结点开始查找
 - 读取3个结点：根结点（51）、中间结点（12 30）、叶结点（15 20 25）
 - 该主码值已经存在，不能插入这条记录



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- 关系模型的参照完整性定义

- 在CREATE TABLE中用**FOREIGN KEY**短语定义哪些列为外码
- 用**REFERENCES**短语指明这些外码参照哪些表的主码



例如，关系SC中一个元组表示一个学生选修的某门课程的成绩，(Sno , Cno) 是主码。Sno , Cno分别参照引用Student表的主码和Course表的主码

【例3】定义SC中的参照完整性

```
CREATE TABLE SC
```

```
(Sno CHAR(9) NOT NULL ,
```

```
Cno CHAR(4) NOT NULL ,
```

```
Grade SMALLINT ,
```

```
PRIMARY KEY (Sno , Cno) , /*在表级定义实体完整性*/
```

```
FOREIGN KEY (Sno) REFERENCES Student(Sno) ,
```

```
/*在表级定义参照完整性*/
```

```
FOREIGN KEY (Cno) REFERENCES Course(Cno)
```

```
/*在表级定义参照完整性*/
```

```
);
```



可能破坏参照完整性的情况及违约处理

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外码值	拒绝
删除元组 →	可能破坏参照完整性	拒绝/级连删除/设置为空值
修改主码值 →	可能破坏参照完整性	拒绝/级连修改/设置为空值



- 参照完整性违约处理

- 1. 拒绝(NO ACTION)执行

- 默认策略

- 2. 级联(CASCADE)操作

- 3. 设置为空值 (SET-NULL)

- 对于参照完整性，除了应该定义外码，还应定义**外码列是否允许空值**



【例4】显式说明参照完整性的违约处理示例

CREATE TABLE SC

(Sno CHAR(9) NOT NULL ,

Cno CHAR(4) NOT NULL ,

Grade SMALLINT ,

PRIMARY KEY (Sno , Cno) ,

FOREIGN KEY (Sno) REFERENCES Student(Sno)

ON DELETE CASCADE /*级联删除SC表中相应的元组*/

ON UPDATE CASCADE , /*级联更新SC表中相应的元组*/

FOREIGN KEY (Cno) REFERENCES Course(Cno)

ON DELETE NO ACTION

/*当删除course 表中的元组造成了与SC表不一致时拒绝删除*/

ON UPDATE CASCADE

/*当更新course表中的cno时 , 级联更新SC表中相应的元组*/);



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



数据库完整性



- 用户定义的完整性是：针对**某一具体应用**的数据必须满足的语义要求
- 关系数据库管理系统提供了定义和检验用户定义完整性的机制，不必由应用程序承担



一、属性上的约束条件定义

- **CREATE TABLE时定义**

- **列值非空 (NOT NULL)**
- **列值唯一 (UNIQUE)**
- **检查列值是否满足一个布尔表达式 (CHECK)**



- 1.不允许取空值

【例5】在定义SC表时，说明Sno、Cno、Grade属性不允许取空值。

```
CREATE TABLE SC
```

```
( Sno CHAR(9) NOT NULL ,
```

```
  Cno CHAR(4) NOT NULL ,
```

```
  Grade SMALLINT NOT NULL ,
```

```
  PRIMARY KEY (Sno , Cno) ,
```

```
  /* 如果在表级定义实体完整性，隐含了Sno , Cno不允许取空值，则在列级  
  不允许取空值的定义就不必写了 */
```

```
);
```



• 2.列值唯一

【例6】建立部门表DEPT，要求部门名称Dname列取值唯一，部门编号Deptno列为主码

```
CREATE TABLE DEPT
```

```
(Deptno NUMERIC(2) ,
```

```
Dname CHAR(9) UNIQUE , /*要求Dname列值唯一*/
```

```
Location CHAR(10) ,
```

```
PRIMARY KEY (Deptno)
```

```
) ;
```



- 3. 用CHECK短语指定列值应该满足的条件

【例7】 Student表的Ssex只允许取“男”或“女”。

CREATE TABLE Student

(Sno CHAR(9) PRIMARY KEY ,

Sname CHAR(8) NOT NULL ,

Ssex CHAR(2) CHECK (Ssex IN ('男' , '女')) ,

/*性别属性Ssex只允许取'男'或'女' */

Sage SMALLINT ,

Sdept CHAR(20)

);



一、属性上的约束条件定义

• 属性上的约束条件检查和违约处理

- 插入元组或修改属性的值时，RDBMS检查属性上的约束条件是否被满足
- 如果不满足则操作被拒绝执行



二、元组上的约束条件定义

- 在CREATE TABLE时可以用**CHECK**短语定义元组上的约束条件，即**元组级的限制**
- 同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件



【例8】当学生的性别是男时，其名字不能以Ms.打头。

CREATE TABLE Student

(Sno CHAR(9) ,

Sname CHAR(8) NOT NULL ,

Ssex CHAR(2) ,

Sage SMALLINT ,

Sdept CHAR(20) ,

PRIMARY KEY (Sno) ,

CHECK (Ssex='女' OR Sname NOT LIKE 'Ms.%')

/*定义了元组中Sname和 Ssex两个属性值之间的约束条件*/

);

- ✓ 性别是女性的元组都能通过该项检查，因为Ssex= '女' 成立；
- ✓ 当性别是男性时，要通过检查则名字一定不能以Ms.打头



二、元组上的约束条件定义

- 元组上的约束条件检查和违约处理

- 插入元组或修改属性的值时，RDBMS检查元组上的约束条件是否被满足
- 如果不满足则操作被拒绝执行



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- **CONSTRAINT 约束**

CONSTRAINT <完整性约束条件名>

[PRIMARY KEY短语

|FOREIGN KEY短语

|CHECK短语]



数据库完整性



【例9】建立学生登记表Student，要求学号在90000~99999之间，姓名不能取空值，年龄小于30，性别只能是“男”或“女”。

CREATE TABLE Student

(Sno NUMERIC(6)

CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999) ,

Sname CHAR(20)

CONSTRAINT C2 NOT NULL ,

Sage NUMERIC(3)

CONSTRAINT C3 CHECK (Sage < 30) ,

Ssex CHAR(2)

CONSTRAINT C4 CHECK (Ssex IN ('男' , '女')) ,

CONSTRAINT StudentKey PRIMARY KEY(Sno)

);

✓ 在Student表上建立了5个约束条件，包括主码约束（命名为StudentKey）以及C1、C2、C3、C4四个列级约束。



数据库完整性



【例10】修改表Student中的约束条件，要求学号改为在900000~999999之间，年龄由小于30改为小于40

可以先删除原来的约束条件，再增加新的约束条件

ALTER TABLE Student

DROP CONSTRAINT C1;

ALTER TABLE Student

ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999) ,

ALTER TABLE Student

DROP CONSTRAINT C3;

ALTER TABLE Student

ADD CONSTRAINT C3 CHECK (Sage < 40) ;

使用ALTER TABLE语句修改表中的完整性限制



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- SQL支持域的概念，并可以用**CREATE DOMAIN**语句建立一个域以及该域应该满足的完整性约束条件。

【例11】建立一个性别域，并声明性别域的取值范围

```
CREATE DOMAIN GenderDomain CHAR(2)
```

```
CHECK (VALUE IN ('男', '女'));
```

这样 [例10] 中对Ssex的说明可以改写为

```
Ssex GenderDomain
```

【例12】建立一个性别域 GenderDomain，并对其中的限制命名

```
CREATE DOMAIN GenderDomain CHAR(2)
```

```
CONSTRAINT GD CHECK ( VALUE IN ('男', '女'));
```



- SQL支持域的概念，并可以用**CREATE DOMAIN**语句建立一个域以及该域应该满足的完整性约束条件。

【例13】删除域GenderDomain的限制条件GD。

```
ALTER DOMAIN GenderDomain  
DROP CONSTRAINT GD;
```

【例14】在域GenderDomain上增加限制条件GDD。

```
ALTER DOMAIN GenderDomain  
ADD CONSTRAINT GDD CHECK (VALUE IN ( '1' , '0' ) );
```

通过 [例13] 和 [例14] ，就把性别的取值范围由('男' , '女')改为 ('1' , '0')



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- 断言是一个谓词，它表达了我们希望数据库总能满足的一个条件。**域约束和参照完整性约束是断言的特殊形式。**

断言创建以后，任何对断言中所涉及的关系的操作都会触发关系数据库管理系统对断言的检查，任何使断言不为真值的操作都会被拒绝执行

- SQL中，可以使用 **CREATE ASSERTION** 语句，通过声明性断言来指定更具一般性的约束。
- 可以定义涉及多个表的或聚集操作的比较复杂的完整性约束。
- SQL Server 2008 及之前版本不支持“断言”



1. 创建断言的语句格式

- **CREATE ASSERTION <断言名> <CHECK 子句>**
- **每个断言都被赋予一个名字，<CHECK 子句>中的约束条件与WHERE子句的条件表达式类似。**

【例15】限制数据库课程最多60名学生选修

```
CREATE ASSERTION ASSE_SC_DB_NUM
```

```
CHECK (60 >= (select count(*)
```

```
/*此断言的谓词涉及聚集操作count的SQL语句*/
```

```
From Course,SC
```

```
Where SC.Cno=Course.Cno and
```

```
Course.Cname = '数据库')
```

```
);
```



【例16】限制每一门课程最多60名学生选修

CREATE ASSERTION ASSE_SC_CNUM1

CHECK(60 >= ALL (SELECT count(*) FROM SC GROUP by cno));

/*此断言的谓词，涉及聚集操作count 和分组函数group by 的SQL语句*/

【例17】限制每个学期每一门课程最多60名学生选修

首先需要修改SC表的模式，增加一个“学期 (TERM)” 属性

ALTER TABLE SC ADD TERM DATE;

然后，定义断言：

CREATE ASSERTION ASSE_SC_CNUM2

**CHECK(60 >= ALL (SELECT count(*)
FROM SC GROUP by cno,TERM));**



2. 删除断言的语句格式为

- **DROP ASSERTION <断言名>;**
- **如果断言很复杂，则系统在检测和维护断言的开销较高，这是在使用断言时应该注意的**



数据库完整性



5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

5.5 域中的完整性限制

5.6 断言

5.7 触发器



- 触发器 (Trigger) 是用户定义在关系表上的一类由**事件驱动** (包括插入、删除和更新) 的特殊过程。

- 触发器保存在数据库服务器中
- 任何用户对表的增、删、改操作均由服务器自动激活相应的触发器
- 触发器可以实施更为复杂的检查和操作，具有更精细和更强大的数据控制能力。
- 检测和维护触发器需要很大的开销，降低了数据库增、删、改的效率！

■ Create Table中的表约束和列约束基本上都是静态的约束，也基本上都是对单一系列或单一元组的约束(尽管有参照完整性)，为实现动态约束以及多个元组之间的完整性约束，就需要触发器技术**Trigger**

■ **Trigger**是一种过程完整性约束(相比之下，Create Table中定义的都是非过程性约束)，是一段程序，该程序可以在特定的时刻被自动触发执行，比如在一次更新操作之前执行，或在更新操作之后执行。



• 触发器的作用

- 防止非法的数据库操纵、维护数据库安全
- 对数据库的操作进行审计，存储历史数据
- 完成数据库初始化处理
- 控制数据库的数据完整性
- 进行相关数据的修改
- 完成数据复制
- 自动完成数据库统计计算
- 限制数据库操作的时间、权限等，控制实体的安全性。

• 设置触发器机制必须满足的两个条件：

- ① 指明什么条件下触发器被执行，即触发条件；
- ② 指明触发器执行的动作是什么，即触发什么。



- **CREATE TRIGGER语法格式**

CREATE TRIGGER <触发器名>

{BEFORE | AFTER} <触发事件> ON <表名>

REFERENCING NEW|OLD ROW AS<变量>

FOR EACH {ROW | STATEMENT}

[WHEN <触发条件>]<触发动作体>

触发器又叫做**事件-条件-动作**（ event-condition-action ）规则。

当特定的系统事件发生时，对规则的条件进行检查，如果条件成立则执行规则中的动作，否则不执行该动作。规则中的动作体可以很复杂，通常是一段**SQL存储过程**。



• 定义触发器的语法说明

(1)只有表的拥有者，即创建表的用户才可以在表上创建触发器，并且一个表上只能创建一定数量的触发器。触发器的具体数量由具体的关系数据库管理系统在设计时确定。

(2)触发器名

- 触发器名可以包含模式名，也可以不包含模式名。同一模式下，触发器名必须是唯一的，并且触发器名和表名必须在同一模式下。

(3)表名

- 触发器只能定义在基本表上，不能定义在视图上。
- 当基本表的数据发生变化时，将激活定义在该表上相应触发事件的触发器，因此该表也称为触发器的目标表。



定义触发器的语法说明

(4)触发事件

- 触发事件可以是INSERT、DELETE或UPDATE,也可以是这几个事件的组合,如INSERT OR DELETE等
- 还可以是UPDATE OF<触发列, ...>,即进一步指明修改哪些列时激活触发器。
- AFTER/BEFORE是触发的时机。AFTER表示在触发事件的操作执行之后激活触发器; BEFORE表示在触发事件的操作执行之前激活触发器。

(5)触发器类型

触发器按照所触发动作的间隔尺寸可以分为**行级触发器**(FOR EACH ROW)和**语句级触发器**(FOR EACH STATEMENT)。



定义触发器的语法说明

(6)触发条件

- 触发器被激活时，只有当触发条件为真时触发动作体才执行，否则触发动作体不执行。
- 如果省略WHEN触发条件，则触发动作体在触发器激活后立即执行。

(7)触发动作体

- 触发动作体既可以是一个匿名PL/SQL过程块，也可以是对已创建存储过程的调用。
- 如果是行级触发器，用户可以在过程体中使用NEW和OLD引用UPDATE/INSERT事件之后的新值和UPDATE/DELETE事件之前的旧值；
- 如果是语句级触发器，则不能在触发动作体中使用NEW或OLD进行引用。
- 如果触发动作体执行失败，激活触发器的事件(即对数据库的增、删、改操作)就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生变化。



【例18】 当对表SC的Grade属性进行修改时，若分数增加了10%则将此次操作记录到下面表中：

SC_U (Sno,Cno,Oldgrade,Newgrade)

其中Oldgrade是修改前的分数，Newgrade是修改后的分数。

CREATE TRIGGER SC_T

AFTER UPDATE OF Grade ON SC

REFERENCING

OLD row AS OldTuple,

NEW row AS NewTuple

FOR EACH ROW

WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)

INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)

VALUES(OldTuple.Sno,OldTuple.Cno,OldTuple.Grade,NewTuple.Grade)



【例19】将每次对表Student的插入操作所增加的学生个数记录到表StudentInsertLog中。

CREATE TRIGGER Student_Count

AFTER INSERT ON Student

/*指明触发器激活的时间是在执行INSERT后*/

REFERENCING

NEW TABLE AS DELTA

FOR EACH STATEMENT

/*语句级触发器, 即执行完INSERT语句后下面的触发动作体才执行一次*/

INSERT INTO StudentInsertLog (Numbers)

SELECT COUNT(*) FROM DELTA



**【例20】 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则
“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”。**

```
CREATE TRIGGER Insert_Or_Update_Sal
  BEFORE INSERT OR UPDATE ON Teacher
  REFERENCING  NEW row AS newTuple
      /*触发事件是插入或更新操作*/
  FOR EACH ROW          /*行级触发器*/
  BEGIN                  /*定义触发动作体，是PL/SQL过程块*/
    IF (newTuple.Job='教授') AND (newTuple.Sal < 4000)
      THEN newTuple.Sal :=4000;
    END IF;
  END;
```



激活触发器

- 触发器的执行，是由**触发事件激活**的，并由数据库服务器自动执行
- 一个数据表上可能定义了**多个触发器**，遵循如下的执行顺序：
 - (1) 执行该表上的BEFORE触发器;
 - (2) 激活触发器的SQL语句;
 - (3) 执行该表上的AFTER触发器。



删除触发器

- 删除触发器的SQL语法：

DROP TRIGGER <触发器名> ON <表名>;

- 触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除。

[例] 删除成绩表SC上的触发器chggrade 。

DROP TRIGGER chggrade ON SC;